AWS CLOUD INFRASTRUCTURE DOCUMENTATION

Infrastructure as Code using Terraform

HashiCorp

Terraform

NAVEEN

TABLE OF CONTENTS

Parameter	Value / Description	Page
Module 1	VPC & Networking — VPC, Subnets, Route Tables, IGW, Elastic IP, NAT Gateway	4
Module 2	Compute Instances — App EC2 (Ubuntu + SSM) & Bastion Host (merged)	8
Module 3	Database — RDS PostgreSQL in private subnet	11
Module 4	Load Balancer — ALB with Target Group attachment	13
Module 5	Storage — S3 Buckets with random globally unique names	16
Module 6	CDN — CloudFront Distribution for Frontend S3	19
Module 7	Secrets Manager — Random password + 2 Secrets	21
Module 8	Terraform Project Structure — main.tf, outputs.tf, variables.tf, state files	24
Module 9	Git — Pushing infrastructure code to a remote repository	29

NAVEEN

1. Infrastructure Overview

This document provides comprehensive technical documentation for the AWS cloud infrastructure provisioned and managed using Terraform. The infrastructure follows a secure, highly available architecture designed with separation of concerns — public-facing resources in public subnets and sensitive backend resources in private subnets.

1.1 Architecture Summary

The infrastructure spans across multiple Availability Zones within a single AWS VPC. Key architectural principles include:

- Network isolation: Public and private subnet segregation
- Secure access: Bastion host for administrative access; SSM for instance management
- High availability: ALB distributing traffic across application instances
- Static content delivery: CloudFront CDN backed by S3 for the frontend
- Secret management: AWS Secrets Manager for sensitive configuration
- Managed database: RDS in a fully private subnet
- Elastic IP allocation ensures a static, predictable outbound IP for all private subnet traffic exiting via the NAT Gateway — useful for IP whitelisting with third-party APIs
- Defence-in-depth approach: public internet → ALB → private EC2 → private RDS, with a security group gate at every hop
- Zero open SSH ports on application infrastructure — SSM Session Manager provides auditable, IAM-controlled shell access without key management overhead
- All sensitive credentials (DB password, app secrets) are injected at runtime from Secrets Manager; no secrets exist in code, environment variables, or Terraform state in plaintext
- S3 + CloudFront offloads all static asset delivery from EC2, reducing compute load and bandwidth costs
- All infrastructure is codified in Terraform — every resource change is a code review, providing a full audit trail via Git history

NAVEEN

Module 1: VPC & Networking

2.1 Overview

The Virtual Private Cloud (VPC) forms the foundational network layer of the entire infrastructure. All AWS resources are deployed within this VPC, providing complete network isolation and control over the IP address space, routing, and access controls.

2.2 VPC Configuration

Parameter	Value / Description
Resource Type	AWS VPC
Terraform Resource	aws_vpc
CIDR Block	10.0.0.0/16
DNS Hostnames	Enabled (Assigns DNS names to EC2 instances.)
DNS Resolution	Enabled (Allows resources inside the VPC to resolve domain names → IP addresses.)
Region	Configured via Terraform variables (var.aws_region)

2.3 Subnets

The VPC is divided into four subnets across two Availability Zones — two public and two privates — ensuring high availability and fault tolerance.

2.3.1 Public Subnets

Parameter	Value / Description
Public Subnet 1	10.0.1.0/24 — Availability Zone A
Public Subnet 2	10.0.2.0/24 — Availability Zone B
Map Public IP	Enabled (auto-assign public IPv4)
Purpose	Bastion Host, Application Load Balancer
Terraform Resource	aws_subnet (public)

2.3.2 Private Subnets

Parameter	Value / Description
Private Subnet 1	10.0.3.0/24 — Availability Zone A
Private Subnet 2	10.0.4.0/24 — Availability Zone B

Parameter	Value / Description
Map Public IP	Disabled (no public IPv4)
Purpose	Application EC2, RDS Database
Terraform Resource	aws_subnet (private)

2.4 Elastic IP (EIP) for NAT Gateway

An Elastic IP (EIP) is provisioned and statically associated with the NAT Gateway. This provides a fixed, predictable public IP address for all outbound traffic originating from private subnet resources.

Parameter	Value / Description
Terraform Resource	aws_eip
Domain	vpc (required for VPC-scoped EIP)
Purpose	Attached to NAT Gateway as its fixed public IP
Allocation	Allocated automatically by AWS; remains yours until released
Associated With	aws_nat_gateway.main (via allocation_id attribute)
Depends On	aws_internet_gateway.main — IGW must exist before EIP is usable in VPC
Cost Note	EIPs are free while associated; charged if unattached — always destroy when not in use

Terraform snippet:

```
resource "aws_eip" "nat" {
  domain     = "vpc"
  depends_on = [aws_internet_gateway.main]

  tags = {
    Name = "${var.project_name}-nat-eip"
  }
}
```

```
resource "aws_nat_gateway" "main" {
  allocation_id = aws_eip.nat.id          # <- EIP attached here
  subnet_id     = aws_subnet.public[0].id  # deployed in Public Subnet 1
  depends_on    = [aws_internet_gateway.main]

  tags = { Name = "${var.project_name}-nat-gw" }
}
```

2.5 Internet & NAT Gateway

Parameter	Value / Description
Internet Gateway	Attached to VPC for public subnet internet access
NAT Gateway	Deployed in Public Subnet 1, uses the EIP above
NAT Purpose	Allows private subnet instances to initiate outbound internet traffic (updates, APIs)
Terraform Resources	aws_internet_gateway, aws_nat_gateway, aws_eip

2.6 Route Tables

Parameter	Value / Description
Public Route Table	Route 0.0.0.0/0 → Internet Gateway; Associated with Public Subnet 1 & 2
Private Route Table	Route 0.0.0.0/0 → NAT Gateway; Associated with Private Subnet 1 & 2
Terraform Resources	aws_route_table, aws_route_table_association

2.7 Security Groups

Security groups act as virtual firewalls for each resource layer:

- ALB Security Group: Allows inbound HTTP (80) and HTTPS (443) from 0.0.0.0/0
- App EC2 Security Group: Allows inbound traffic only from ALB SG on app port; outbound all
- Bastion Security Group: Allows inbound SSH (22) from trusted IPs only; outbound all
- RDS Security Group: Allows inbound on DB port only from App EC2 SG

Module 2: Compute Instances — App EC2 & Bastion Host

Both EC2 instances — the Application Server and the Bastion Host — are documented together in this module as both are Ubuntu-based compute resources provisioned via the aws_instance resource. They differ in subnet placement, purpose, and access method.

2A. Application EC2 Instance

Overview

The Application EC2 instance hosts the backend application and runs in a private subnet to prevent direct public internet exposure. All traffic to the application is routed through the Application Load Balancer. The instance uses Ubuntu as the operating system and has AWS Systems Manager (SSM) capability for secure, keyless access.

EC2 Configuration

Parameter	Value / Description
Terraform Resource	aws_instance (logical name: app)
AMI	Ubuntu 22.04 LTS — fetched dynamically via aws_ami data source
Operating System	Ubuntu Linux 22.04
Subnet	Private Subnet 1 (10.0.3.0/24)
Public IP	Disabled — no public IP assigned
Instance Type	Configurable via Terraform variable (e.g., t3. medium)
Root Volume	EBS gp3, configurable size (e.g., 50 GiB)
Key Pair	Optional — SSM Session Manager is the preferred access method

IAM Role & SSM Integration

The EC2 instance has an IAM Instance Profile attached, enabling AWS Systems Manager (SSM) Session Manager access. This eliminates the need to expose SSH ports or manage SSH keys.

Parameter	Value / Description
IAM Role	ec2-ssm-role (custom)
Managed Policy	AmazonSSMManagedInstanceCore

Parameter	Value / Description
Instance Profile	Attached to EC2 via iam_instance_profile argument
Terraform Resources	aws_iam_role, aws_iam_role_policy_attachment, aws_iam_instance_profile
Access Method	AWS Console → Systems Manager → Session Manager OR aws ssm start-session --target <instance-id>

Security

- Instance resides in private subnet — not directly reachable from the internet
- Security group restricts inbound traffic to ALB security group only
- SSM role allows session-based shell access without exposing SSH port
- Outbound traffic exits via NAT Gateway for package updates and external API calls

2B. Bastion Host

Overview

The Bastion Host is a hardened EC2 instance deployed in a public subnet. It serves as the secure jump server for administrators who require direct SSH access to private subnet resources such as the RDS database or the application EC2 instance in scenarios where SSM is insufficient.

Bastion Configuration

Parameter	Value / Description
Terraform Resource	aws_instance (logical name: bastion)
AMI	Ubuntu 22.04 LTS
Subnet	Public Subnet 1 (10.0.1.0/24)
Public IP	Enabled — Elastic IP or auto-assigned public IP
Instance Type	t3. micro (minimal resources needed)
Key Pair	SSH Key Pair — required; dedicated bastion key pair
Security Group	Bastion SG — allows SSH (22) from trusted CIDR blocks only

Access Pattern

Recommended admin access flow:

- Step 1: SSH to Bastion Host using private key from an allowed IP
  - ssh -i keypair.pem ubuntu@<bastion-public-ip>

- Step 2: From Bastion, SSH to private EC2 or connect to RDS
  - ssh -i keypair.pem ubuntu@<private-ec2-ip>
  - psql -h <rds-endpoint> -U admin -d mydb
- Step 3: Terminate sessions after use — never leave Bastion sessions idle

Security Best Practices

- Restrict SSH inbound CIDR to specific office or VPN IP addresses only
- Use a dedicated key pair for bastion — do not reuse application keys
- Consider scheduled auto-stop for the Bastion outside business hours
- Enable CloudWatch logging for all SSH sessions
- Do not install application dependencies on the Bastion — keep it minimal

Module 3: RDS Database

3.1 Overview

The RDS instance is deployed in a private subnet with no public accessibility. This ensures the database is never exposed to the internet and can only be reached from within the VPC — specifically from application EC2 instances and from the Bastion when tunneling is needed.

3.2 RDS Configuration

Parameter	Value / Description
Terraform Resource	aws_db_instance
Subnet Group	aws_db_subnet_group — spans Private Subnet 1 & 2
Subnet	Private Subnet 1 & 2 (multi-AZ failover support)
Publicly Accessible	false
Engine	PostgreSQL (configurable via Terraform variable)
Engine Version	Configurable via Terraform variable
Instance Class	db.t3. micro (configurable)
Storage Type	gp3
Allocated Storage	100 GiB (configurable)
Max Alloc. Storage	500 GiB — auto-scaling = allocated storage × 5 (100 × 5 = 500 GiB)
Backup Retention	7 days — automated snapshots
Multi-AZ	Configurable — true for production, false for dev/staging
Password	Fetched directly from Secrets Manager at creation time (see Module 7)

3.3 Credentials Management

Database credentials are not hardcoded in Terraform. The password is read directly from AWS Secrets Manager using a data source and a local value, then passed to the RDS resource:

```
# 1. Fetch the secret from Secrets Manager
data "aws_secretsmanager_secret_version" "rds_creds" {
  secret_id = aws_secretsmanager_secret.db_credentials.id
}
```

```
# 2. Decode JSON and expose as a local
locals {
  rds_creds = jsondecode(
    data.aws_secretsmanager_secret_version.rds_creds.secret_string
  )
}

# 3. Use in RDS resource — password comes from the secret
resource "aws_db_instance" "main" {
  username = local.rds_creds.username
  password = local.rds_creds.password   # <- fetched from Secrets Manager
  # ... other config
}
```

- DB username and password stored in Secrets Manager (see Module 7)
- Application also retrieves credentials at runtime via AWS SDK
- IAM-based RDS authentication can be layered on top for additional security

3.4 Security

- RDS Security Group allows inbound only on the DB port from App EC2 Security Group
- Bastion can connect via SSH tunnel for DBA access when needed
- No public endpoint — connection only possible within the VPC
- Encryption at rest enabled via AWS KMS
- Automated backups and snapshot retention configured

Module 4: Application Load Balancer (ALB)

4.1 Overview

The Application Load Balancer (ALB) serves as the entry point for all web traffic to the application. It distributes incoming HTTP/HTTPS requests to the backend EC2 instances in private subnets, providing high availability, SSL termination, and health checking.

4.2 ALB Configuration

Parameter	Value / Description
Terraform Resource	aws_lb
Load Balancer Type	application
Scheme	internet-facing
Subnets	Public Subnet 1 & Public Subnet 2 (two AZs required)
Security Group	ALB SG — allows 80 (HTTP) and 443 (HTTPS) from 0.0.0.0/0
Access Logs	Stored in the application S3 bucket

4.3 Target Group Configuration

Parameter	Value / Description
Terraform Resource	aws_lb_target_group
Target Type	instance
Protocol	HTTP
Port	Application port (e.g., 3000 — as per your app)
VPC	Attached to the same VPC as the EC2 instance
Health Check Path	/health or / (configurable)
Health Check Interval	30 seconds
Healthy Threshold	2
Unhealthy Threshold	3

4.4 Attaching Target Group to the ALB

The target group is linked to the ALB in two steps: (1) registering the EC2 instance as a target, and (2) configuring an ALB Listener that forwards traffic to the target group.

Step 1 — Register EC2 Instance to Target Group

```
resource "aws_lb_target_group_attachment" "app" {
  target_group_arn = aws_lb_target_group.app.arn
  target_id        = aws_instance.app.id   # App EC2 instance ID
  port             = 3000                   # Port your app listens on or you can
have variable for this
}
```

Step 2 — ALB Listener (HTTP → forward to target group)

```
resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.main.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.app.arn
  }
}
```

Step 3 — ALB Listener (HTTPS with ACM Certificate)

```
resource "aws_lb_listener" "https" {
  load_balancer_arn = aws_lb.main.arn
  port              = 443
  protocol          = "HTTPS"
  ssl_policy        = "ELBSecurityPolicy-TLS13-1-2-2021-06"
  certificate_arn   = aws_acm_certificate.main.arn

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.app.arn
  }
}
```

Key points on target group attachment:

- The target group must be in the same VPC as the ALB and the EC2 instance
- aws_lb_target_group_attachment links the specific EC2 instance ID to the TG
- The listener default_action type="forward" routes all incoming requests to the TG
- ALB continuously health-checks the registered targets; unhealthy targets are removed from rotation automatically
- Multiple instances can be registered to the same TG for horizontal scaling

4.5 DNS & SSL

- ALB DNS name is used as the CNAME target for the application domain
- ACM (AWS Certificate Manager) certificate attached to the HTTPS listener

- HTTP to HTTPS redirect rule can be configured on the port 80 listener

Module 5: S3 Buckets

5.1 Overview

Two S3 buckets are provisioned: one dedicated to hosting frontend static assets (integrated with CloudFront) and one for application-level storage such as uploads, logs, or artifacts. S3 bucket names must be globally unique across all AWS accounts — this is enforced using Terraforms random_id resource to append a unique hex suffix.

5.2 Generating Globally Unique Bucket Names with random_id

S3 bucket names are globally unique across all AWS accounts and regions. Hardcoding a name risks collision. The Terraform random provider solves this by generating a stable random hex suffix:

```
# Terraform random provider — add to versions.tf or provider.tf
terraform {
  required_providers {
    random = {
      source  = "hashicorp/random"
      version = "~> 3.0"
    }
  }
}

# Generate a random 4-byte hex suffix (e.g., "a3f9c1b2")
resource "random_id" "bucket_suffix" {
  byte_length = 4    # 4 bytes = 8 hex characters
}

# Frontend bucket — globally unique name
resource "aws_s3_bucket" "frontend" {
  bucket = "${var.project_name}-frontend-${random_id.bucket_suffix.hex}"
  # e.g., "myapp-frontend-a3f9c1b2"
}

# Storage/App bucket — globally unique name
resource "aws_s3_bucket" "storage" {
  bucket = "${var.project_name}-storage-${random_id.bucket_suffix.hex}"
  # e.g., "myapp-storage-a3f9c1b2"
}
```

Key notes on random_id usage:

- byte_length = 4 produces 8 hex characters — enough entropy for uniqueness
- The suffix is stable after first apply — Terraform stores it in state and reuses it
- Both buckets share the same suffix, so names are consistent and easy to identify
- random_id.bucket_suffix.hex outputs lowercase hex; random_id.bucket_suffix.b64_url is also available

- Never run terraform taint on random_id in production — it will regenerate the suffix and rename the buckets

5.3 Frontend S3 Bucket

Parameter	Value / Description
Terraform Resource	aws_s3_bucket (logical name: frontend)
Bucket Name	${var.project_name}-frontend-${random_id.bucket_suffix.hex}
Purpose	Hosts React / Angular / Vue static build files
Public Access	Fully blocked — access only via CloudFront OAC
Versioning	Enabled
Static Website	Disabled — CloudFront handles routing and SPA fallback
Bucket Policy	Allows GetObject from CloudFront Origin Access Control only
Encryption	SSE-S3 (or SSE-KMS)
Lifecycle Rules	Optional — auto-expire old non-current versions after N days

5.4 Application / Storage S3 Bucket

Parameter	Value / Description
Terraform Resource	aws_s3_bucket (logical name: storage)
Bucket Name	${var.project_name}-storage-${random_id.bucket_suffix.hex}
Purpose	Application uploads, logs, backups, ALB access logs
Public Access	Fully blocked — private bucket
Versioning	Enabled
Access	EC2 instance via IAM role with S3 read/write policy
Encryption	SSE-S3 or SSE-KMS
Terraform Resources	aws_s3_bucket, aws_s3_bucket_public_access_block, aws_s3_bucket_policy

Module 6: CloudFront Distribution

6.1 Overview

Amazon CloudFront is configured as the CDN layer in front of the frontend S3 bucket. It caches and distributes static assets globally from AWS edge locations, reducing latency and improving performance for end users worldwide.

6.2 CloudFront Configuration

Parameter	Value / Description
Terraform Resource	aws_cloudfront_distribution
Origin	Frontend S3 Bucket
Origin Access	Origin Access Control (OAC) — restricts S3 to CloudFront only
Viewer Protocol Policy	redirect-to-https
Allowed HTTP Methods	GET, HEAD (read-only for static assets)
Cache Policy	CachingOptimized (AWS managed)
Default Root Object	index.html
Geo Restriction	None (or configurable)
Alternate Domain	Custom domain — CNAME configured via Route 53 or DNS provider

6.3 Origin Access Control (OAC)

OAC replaces the legacy Origin Access Identity (OAI) and provides more secure, signed-request access from CloudFront to S3:

- S3 bucket policy grants GetObject permission to the CloudFront service principal
- Signed requests ensure only CloudFront can read from the bucket
- S3 public access block remains fully enabled

6.4 Deployment Flow

- Step 1: Build the frontend application (e.g., npm run build)
- Step 2: Upload build artifacts to the Frontend S3 bucket:
  - aws s3 sync ./build s3://<frontend-bucket-name> --delete
- Step 3: Invalidate the CloudFront cache to serve the latest files:
  - aws cloudfront create-invalidation --distribution-id <id> --paths '/*'

Module 7: AWS Secrets Manager & Random Password

7.1 Overview

Two AWS Secrets Manager secrets are provisioned to store sensitive configuration values securely. The database password is auto-generated using Terraforms random password resource, stored in Secrets Manager, and then fetched directly during RDS creation — ensuring the password never appears in source code or tfvars files.

7.2 Generating a Random Password with random password

The random_password resource generates a cryptographically strong password that meets AWS RDS requirements:

```
resource "random_password" "db_password" {
  length           = 20        # Minimum recommended for RDS
  upper            = true      # Include uppercase letters (A-Z)
  lower            = true      # Include lowercase letters (a-z)
  numeric          = true      # Include digits (0-9)
  special          = true      # Include special characters
  override_special = "!#$%&*-_=+?"  # Safe chars for RDS (avoid @, /, \")
  min_upper        = 2
  min_lower        = 2
  min_numeric      = 2
  min_special      = 2
}
```

Password norms enforced:

- Length of 20 characters — well above the RDS minimum of 8
- At least 2 characters from each class (upper, lower, numeric, special)
- Special characters restricted to those safe for RDS connection strings
- Characters avoided: @ / " \\ space — these break JDBC/ODBC DSN strings
- Password is marked sensitive = true in Terraform state — never printed in plan output

7.3 Secret 1 — Database Credentials

Parameter	Value / Description
Terraform Resource	aws_secretsmanager_secret
Secret Name	Follow AWS naming norms: /<env>/<app>/db-credentials e.g., /prod/myapp/db-credentials
Purpose	Stores RDS database username and auto-generated password

Parameter	Value / Description
Secret Format	JSON: {"username": "dbadmin", "password": "<random_password>"}
Password Source	random_password.db_password.result — never hardcoded
Rotation	Automatic rotation recommended (aws_secretsmanager_secret_rotation)
Access	EC2 IAM role granted secretsmanager:GetSecretValue
Encryption	AWS KMS (default aws/secretsmanager key or custom CMK)

```
resource "aws_secretsmanager_secret" "dB credentials" {
  name = "/${var.environment}/${var.project_name}/db-credentials"
}

resource "aws_secretsmanager_secret_version" "db_credentials" {
  secret_id = aws_secretsmanager_secret.db_credentials.id
  secret_string = jsonencode({
    username = var.db_username
    password = random_password.db_password.result  # <- random generated password
  })
}
```

7.4 Fetching Password Directly During RDS Creation

The RDS resource reads the password from the secret using a data source and local value, so the password flows from Secrets Manager → local.rds_creds.password → aws_db_instance:

```
data "aws_secretsmanager_secret_version" "rds_creds" {
  secret_id  = aws_secretsmanager_secret.db_credentials.id
  depends_on = [aws_secretsmanager_secret_version.db_credentials]
}

locals {
  rds_creds = jsondecode(
    data.aws_secretsmanager_secret_version.rds_creds.secret_string
  )
}

resource "aws_db_instance" "main" {
  identifier        = "${var.project_name}-db"
  engine            = "postgres"
  instance_class    = var.db_instance_class
  allocated_storage = 100

  db_name  = var.db_name
  username = local.rds_creds.username
  password = local.rds_creds.password   # <- fetched from Secrets Manager
```

```
  skip_final_snapshot = false
}
```

7.5 Secret 2 — Application Configuration

Parameter	Value / Description
Terraform Resource	aws_secretsmanager_secret
Secret Name	As you wish, but follow AWS norms
Purpose	Stores API keys, third-party service credentials, JWT secrets
Secret Format	JSON key-value pairs for app configuration
Encryption	AWS KMS (default or custom KMS key)

Module 8: Terraform Project Structure

8.1 main.tf — Provider & Version Configuration

The main.tf (or provider.tf / versions.tf) at the root of your project declares the Terraform version constraint, required providers, and the AWS provider configuration:

```
################################################################################
# main.tf
################################################################################
terraform {
  required_version = ">= 1.5"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
    random = {
      source = "hashicorp/random"
      version = "~> 3.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

8.2 outputs.tf — Terraform Outputs

Outputs expose important resource attributes after terraform apply. These are the actual outputs from the project:

```
################################################################################
# outputs.tf
################################################################################

output "alb_dns_name" {
  description = "Application URL — open this in your browser"
  value       = "http://${aws_lb.main.dns_name}"
}

output "bastion_public_ip" {
  description = "Bastion Host public IP"
  value       = aws_instance.bastion.public_ip
}

output "app_ec2_private_ip" {
  description = "App EC2 private IP (only reachable via Bastion)"
  value       = aws_instance.app.private_ip
}

output "rds_endpoint" {
```

```
  description = "RDS PostgreSQL endpoint"
  value       = aws_db_instance.main.address
}

output "rds_db_name" {
  description = "RDS database name"
  value       = aws_db_instance.main.db_name
}

output "frontend_bucket_name" {
  description = "S3 bucket where you upload your frontend code"
  value       = aws_s3_bucket. frontend.bucket
}

output "storage_bucket_name" {
  description = "S3 bucket used for storage"
  value       = aws_s3_bucket. storage.bucket
}

output "cloudfront_domain_name" {
  description = "CloudFront domain name"
  value       = aws_cloudfront_distribution.frontend.domain_name
}
```

You can add other Outputs as you needed, and whatever the information you want in Output

8.3 variables.tf — Input Variable Declarations

variables.tf declares all input variables with their types, descriptions, and optional default values

Example given below: var.aws_region should be used in code whenever we need this value.

```
##############################################################################
# variables.tf
##############################################################################

variable "aws_region" {
  description = "AWS region to deploy resources"
  type        = string
  default     = "us-east-1"}
```

8.4 terraform.tfvars — Variable Values

terraform.tfvars supplies actual values for the declared variables. This file should be .gitignored in production and managed via CI/CD secrets or SSM Parameter Store.

```
# terraform.tfvars
# WARNING: Do NOT commit this file to version control if it contains secrets

aws_region        = "us-east-1"
```

```
project_name      = "myapp"
environment       = "prod"
db_username       = "dbadmin"
db_name           = "appdb"
db_instance_class = "db.t3.micro"
app_instance_type = "t3.medium"
```

8.5 Terraform State Files

Terraform uses several files to track infrastructure state:

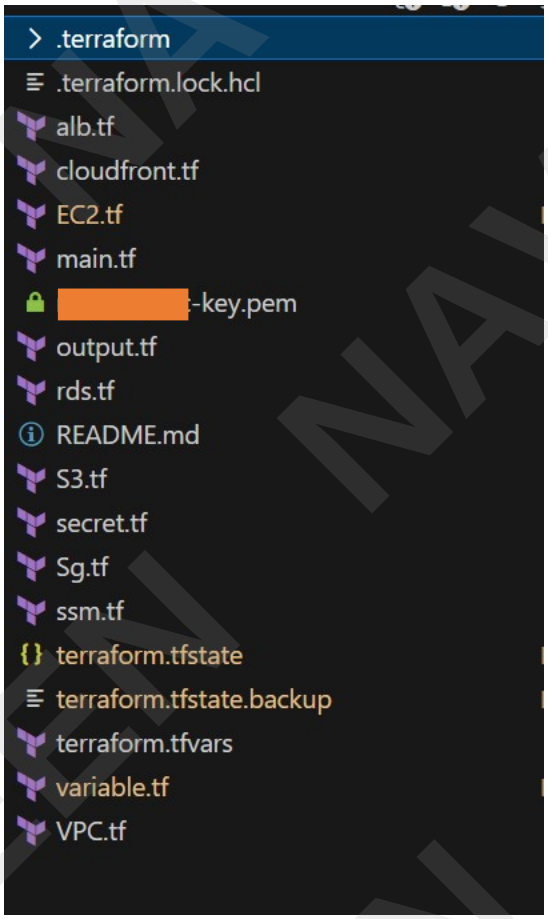
Parameter	Value / Description
terraform.tfstate	The current state file — JSON record of all managed resources and their attributes. Single source of truth for Terraform.
terraform.tfstate.backup	Backup of the previous state — automatically created before every apply. Used for rollback if needed.
.terraform.lock.hcl	Provider lock file — pins exact provider versions (checksums) used in this workspace. Must be committed to Git to ensure consistent provider versions across the team.
.terraform/	Local plugin cache directory — contains downloaded provider binaries after terraform init. Never commit this directory.

State file best practices:

- Use remote state backend (S3 + DynamoDB) for team environments — never share local state files
- Commit.terraform.lock.hcl to Git — this ensures your team uses identical provider versions
- Add terraform.tfstate, terraform.tfstate.backup, and. terraform/ to .gitignore
- Never manually edit terraform.tfstate — use terraform state commands instead
- Enable versioning on the S3 state bucket so you can roll back to any previous state

```
# .gitignore — recommended Terraform entries
.terraform/
terraform.tfstate
terraform.tfstate.backup
*.tfvars          # contains sensitive variable values
*.tfplan          # saved plan files may contain sensitive data
crash.log
override.tf
override.tf.json
*_override.tf
*_override.tf.json
```

8.6 Recommended Directory Layout



8.7 Common Terraform Commands

Parameter	Value / Description
terraform init	Initialize working directory, download providers and modules
terraform plan	Preview changes before applying
terraform apply	Apply changes to provision or update infrastructure
terraform apply -auto-approve	Apply without interactive confirmation (use in CI/CD only)
terraform destroy	Destroy all managed infrastructure
terraform output	Display output values (ALB DNS, endpoints, etc.)
terraform fmt -recursive	Format all .tf files to canonical style
terraform validate	Validate configuration syntax and types

Parameter	Value / Description
terraform state list	List all resources tracked in Terraform state
terraform state show <resource>	Show full details of a specific state resource
terraform refresh	Sync state with real-world infrastructure (read-only)

NAVEEN

Module 9: Pushing Infrastructure Code to Git

9.1 Overview

Version-controlling your Terraform code in Git is a best practice for collaboration, audit trails, and CI/CD pipelines. This module covers initializing a Git repository, configuring .gitignore correctly, and pushing to a remote (GitHub, GitLab, Bitbucket, AWS Code Commit, etc.).

9.2 Pre-Flight Checklist Before First Commit

- Ensure .gitignore includes state files, terraform/, and tfvars files with secrets
- Review all .tf files — confirm no hardcoded secrets, API keys, or passwords
- Confirm.terraform.lock.hcl is present (generated after terraform init)
- Run terraform fmt -recursive to canonically format all files
- Run terraform validate to check for configuration errors.

9.3 Initialize a New Git Repository

```
# Navigate to your infrastructure directory
cd infrastructure/

# Initialize a new Git repository
git init

# Create .gitignore BEFORE your first commit
 Add these files inside it
terraform.tfstate
terraform.tfstate.backup
*.tfvars
*.tfplan
crash.log
EOF
```

9.4 Stage, Commit, and Push

```
# Stage all tracked files (respects .gitignore)
git adds.

# Verify what will be committed — check no secrets are included
git status
git diff --cached

# Create the initial commit
git commit -m "feat: initial Terraform AWS infrastructure"

# Add remote origin (replace with your actual repository URL)
git remote add origin https://github.com/<your-org>/<repo-name>. git

# Push to the main branch (use -u to set upstream tracking)
```

```
git push -u origin main
```

9.5 For Existing Repositories (adding IaC to a mono-repo)

```
# Clone the existing repo
git clone https://github.com/<your-org>/<repo-name>. git
cd <repo-name>

# Create an infra directory and move your files
mkdir -p infra
cp -r /path/to/your/terraform/* infra/

# Stage and commit
git add infra/
git commit -m "feat: add Terraform infrastructure code"
git push origin main
```

9.6 Branch Strategy for Infrastructure

Parameter	Value / Description
main / master	Production-ready, fully tested infrastructure code. Protected branch — no direct push.
develop	Integration branch for in-progress changes. Deployed to staging automatically via CI/CD.
feature/<name>	Short-lived feature branches for new infrastructure components.
hotfix/<name>	Emergency patches to production infrastructure.

9.9 Security — What NOT to Push to Git

- terraform.tfstate / terraform.tfstate.backup — contains all resource attributes including sensitive values
- terraform.tfvars — may contain passwords, API keys, or sensitive variable values
- *.tfplan files — saved plan files can contain sensitive diffs
- . terraform/ directory — binary provider plugins; re-downloaded via terraform init
- Any file with hardcoded AWS access keys, secrets, or passwords

Always run git status and inspect git diff --cached before every commit to confirm no sensitive files are staged.

10. Security Considerations

The following security controls are implemented across the infrastructure:

10.1 Network Security

- All sensitive resources (EC2, RDS) are placed in private subnets with no public IPs
- Security groups implement least-privilege rules — each resource accepts traffic from authorized sources only
- NAT Gateway enables outbound-only connectivity from private subnets
- No 0.0.0.0/0 inbound rules on private resources

10.2 Access Control

- IAM roles follow the least-privilege principle — EC2 only has necessary permissions
- SSM Session Manager eliminates SSH key management for app server access
- Bastion host SSH access restricted to specific office/VPN CIDR only
- RDS access restricted to the application security group

10.3 Data Protection

- S3 buckets have public access blocked at account and bucket level
- CloudFront OAC prevents direct S3 access, bypassing the CDN
- All secrets stored in AWS Secrets Manager — never in code or tfvars
- RDS encryption at rest via AWS KMS
- HTTPS enforced via CloudFront viewer policy and ALB listener redirect

NAVEEN

11. Key Terraform Outputs

The following outputs are defined in outputs.tf and provide essential connection and resource information after terraform apply:

Parameter	Value / Description
alb_dns_name	http://<alb-dns> — Application URL; open in browser
bastion_public_ip	Public IP of the Bastion Host — use for SSH access
app_ec2_private_ip	Private IP of the Application EC2 — reachable only via Bastion
rds_endpoint	RDS PostgreSQL connection endpoint
rds_db_name	RDS database name
frontend_bucket_name	Frontend S3 bucket name (includes random suffix)
storage_bucket_name	Storage S3 bucket name (includes random suffix)
cloudfront_domain_name	CloudFront distribution domain — use as your frontend CDN URL

NAVEEN